

Logistic Regression

A Powerful Tool for Classification Academic Year 1447 Term
1 Dr. Jomana Bashatah Slides adapted from Casten Lange

Basics of Logistic Regression

Classification Algorithm:

- **Two categories for the outcome variable (analysed in what follows):** e.g. Unemployed true or false
- Multiple categories for the outcome variable (not covered here)
 - unordered logistic regression
 - ordered logistic regression »

A Mock-Up Example to Introduce the Idea

- After a few years past graduation, 11 college friends reunite
- Financially, some are doing very well and others are doing just OK
- The table below shows the yearly salary (in thousands of dollars) and if they own a yacht ($Yacht = 1$) or not ($Yacht = 0$)
 - Note: the table only contains 10 rows as one of them (Razan) has not arrived yet. We will talk about Razan later on

Using OLS is a Tempting (but bad) Idea

Using OLS is a Tempting (but bad) Idea

- An obvious - but sub-optimal as we will see later on - approach is a linear regression
- Use OLS to find a regression line with slope β_1 and intercept β_2 that best approximates the data (minimizing the MSE)

Linear Function for Binary Classification

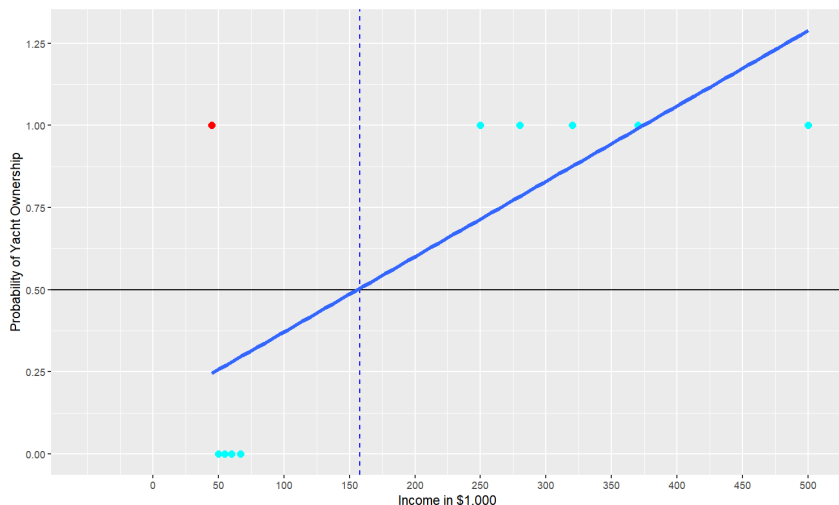
- The underlying regression assigns continuous numbers to different income levels to different income levels rather than assign categorical values such as 1 (“yacht owner: yes”) or 0 (“yacht owner: no”)
- Therefore, we interpret the continuous predictions from the regression as probabilities for being a yacht owner ($Prob_i$)
- The underlying probability is called the Linear Probability Model (LPM)
- The prediction equation would be $prob_i = \beta_1 Inc_i + \beta_2$
- The prediction equation from the LPM can be used to predict the probability of a person being a yacht owner based on their income.
 - Whenever the outcome variable (the probability) is greater than 0.5, we predict “yacht owner: yes”, otherwise, “yacht owner: no”

Example Calculations

$$prob_i = 0.0023 Inc_i + 0.1418$$

- A person has an income of \$75,000
- $prob_i = 0.0023 * 75 + 0.1418$
- $prob_i = 0.3413$
- since $0.3413 < 0.5$, then we predict “yacht owner: no”

Quick Way to Find a Decision Boundary

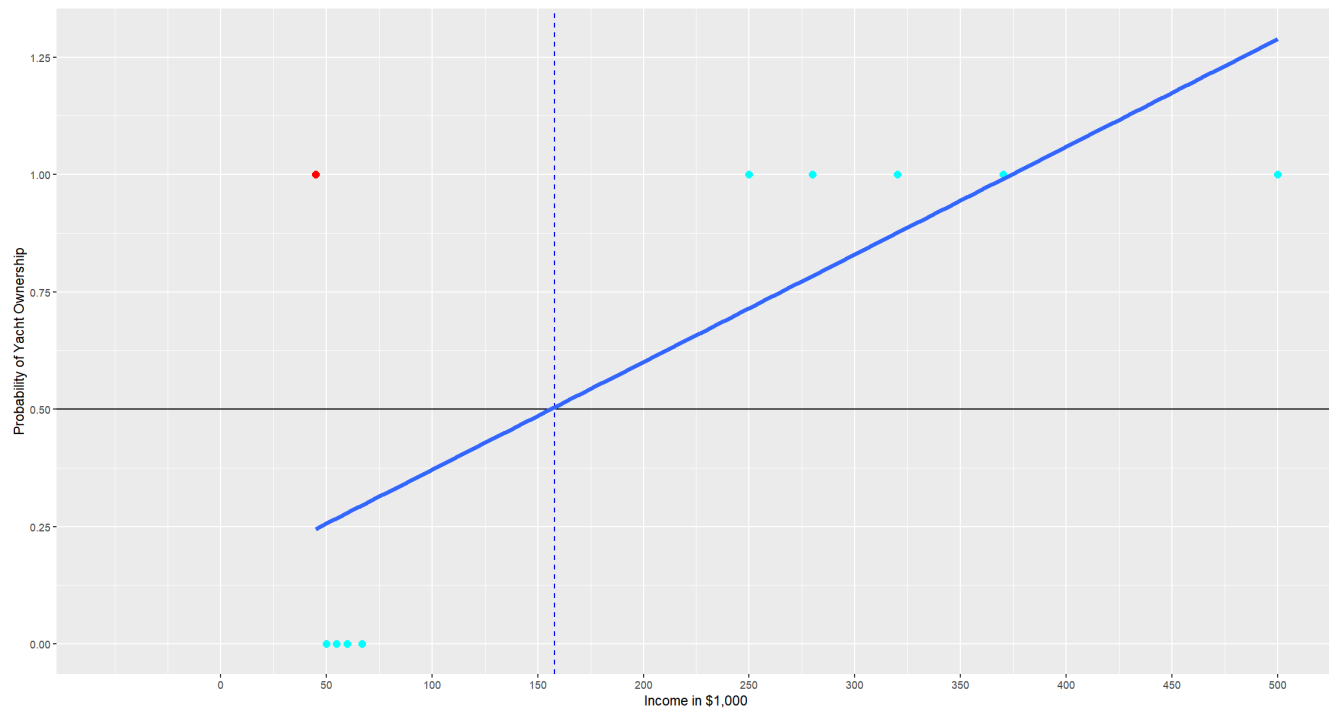


1. Find the intersection point between the prediction line and the horizontal 0.5 probability line.

2. Draw a vertical line through the intersection point. This line is called a **decision boundary**.

3. All incomes left of the *decision boundary* (income smaller than 158) are predicted as “no”. All incomes right of the *decision boundary* (income greater than 158) are predicted as “yes”.

Why OLS for Classification is a Bad Idea



Note, incomes $> \$370,000$ are predicted with a probability $> 100\%$ to be yacht owners(?)

E.g. probability of owning a yacht for an income of $\$500,000$ is 125% (?)

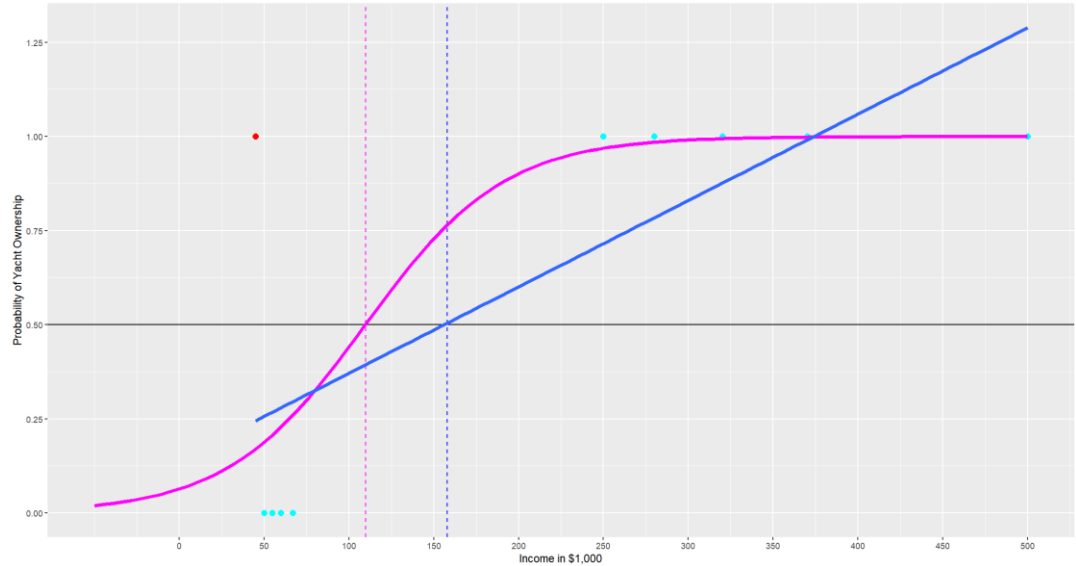
A similar problem can occur with negative probabilities!

A Step-Function as an Alternative to OLS

If we are willing to give up the linearity requirement, we can avoid these problems

A non-linear prediction line can bend for greater income values downwards and thus avoid exceeding 1.

For small income values, the prediction line can bend upwards and thus avoid falling below 0.



Popular Step-Functions (Sigmoid Function)

- Since the curve looks like a step of a staircase (first horizontal, then almost vertical, and finally horizontal again), these types of curves are called step-function.
 - Scientifically, they are called `sigmoid functions`
- Sigmoid Functions Include
 - The **Hyperbolic Tangent function**.
 - The **Arc Tangent function**.
 - The **Logistic function** .

The Logistic Function

- The **Logistic function** (confusingly sometimes also called the *sigmoid function*):

$$y_i = \frac{1}{1 + e^{-x_i}}$$

We use: $y_i = P_{yes,i}^{prob}$ and $x_i = \beta_1 Inc_i + \beta_2$ which gives us:

$$P_{yes,i}^{prob} = \frac{1}{1 + e^{-(\beta_1 Inc_i + \beta_2)}}$$

β_1 and β_2 change slope and position

β_2 shifts the curve horizontally

β_1 determines the steepness of the step

See: [Try it yourself](#)

What Makes the Logistic Function so Special?

compared to other sigmoid (step) functions

Time for some mathematical magic:

Logistic function $P_{yes,i}^{rob}$:= probability for positive event (e.g. yacht ownership: yes):

$$P_{yes,i}^{rob} = \frac{1}{1 + e^{-(\beta_1 \cdot x_i + \beta_2)}}$$

Take the inverse on both sides of the equation:

$$\frac{1}{P_{yes,i}^{rob}} = 1 + e^{-(\beta_1 \cdot x_i + \beta_2)}$$

Subtract 1 on both sides:

$$\frac{1}{P_{yes,i}^{rob}} - 1 = e^{-(\beta_1 \cdot x_i + \beta_2)}$$

Consider that $-1 = -\frac{P_{yes,i}^{rob}}{P_{yes,i}^{rob}}$ and substitute -1 accordingly, we get after simplification:

$$\frac{1 - P_{yes,i}^{rob}}{P_{yes,i}^{rob}} = e^{-(\beta_1 \cdot x_i + \beta_2)}$$

$1 - P_{yes,i}^{prob}$ equals by definition $P_{no,i}^{prob}$:

$$\frac{P_{no,i}^{prob}}{P_{yes,i}^{prob}} = e^{-(\beta_1 \cdot x_i + \beta_2)}$$

Take again the inverse on both sides:

$$\frac{P_{yes,i}^{prob}}{P_{no,i}^{prob}} = e^{\beta_1 \cdot x_i + \beta_2}$$

Take the logarithm on both sides:

$$\ln\left(\frac{P_{yes,i}^{prob}}{P_{no,i}^{prob}}\right) = \beta_1 \cdot x_i + \beta_2$$

One More Step — Odds vs Probabilities

- The fraction of the yes/no probabilities can be interpreted as *Odds* as they are often used in betting.
- Example: The probability of getting two heads when flipping two coins is $P_{yes,i}^{prob} = 0.25$.
- Consequently, the probability of **not** getting two heads when flipping two coins is $P_{no,i}^{prob} = 0.75$.
- *Odds* for 2 Heads compared to **not** 2 heads is 1 to 3 or 33%:

...

$$O^{dds} = \frac{P_{yes,i}^{prob}}{P_{no,i}^{prob}} = \frac{0.25}{0.75} = \frac{1}{3} = 0.33$$

Interpretation of the β s: Yacht Ownership

$$\ln(O^{dds}) = \ln\left(\frac{p_{yes,i}^{prob}}{p_{no,i}^{prob}}\right) = 0.02 \cdot Inc_i + (-2.7)$$

Model results after running and printing the workflow():

```
# The workflow WFYachts had been created before and  
# DataTest were fitted to the workflow.
```

```
print(WFYachts)
```

```
== Workflow [trained] ==
```

```
Preprocessor: Recipe
```

```
Model: logistic_reg()
```

```
— Preprocessor —
```

```
0 Recipe Steps
```

```
— Model —
```

```
Call: stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

```
Coefficients:
```

```
(Intercept)      Income  
-2.68660      0.02448
```

```
Degrees of Freedom: 9 Total (i.e. Null); 8 Residual
```

```
Null Deviance:      13.46
```

```
Residual Deviance: 5.654    AIC: 9.654
```

Interpretation of the β s: Yacht Ownership

$$\ln(Odds) = \ln\left(\frac{P_{yes,i}^{prob}}{P_{no,i}^{prob}}\right) = 0.02 \cdot Inc_i + (-2.7)$$

- If income increases by 1 (\$1,000) the logarithm of the odds increases by 0.02.
- Since change of a logarithm is a relative change (**percentage**):

...

If income increases by 1 (\$1,000) the odds increases by 2% (0.02).
(careful with the results because data were made up and N is too small!)

Confusion Matrix

Note, in the mockup we did not create training and testing data. Therefore, we use DataYachts (the data we used to fit/train the workflow) here. This is not a proper methodology but good enough for the mock-up:

```
DataYachtsWithPred=augment (WFYachts,  
new_data=DataYachts)  
conf_mat(DataYachtsWithPred, truth=Yacht,  
estimate=.pred_class)
```

	Truth	
Prediction	0	1
0	4	1
1	0	5

Real World Churn Analysis with Logistic Regression

— the Data

We use data (7,043 customers) of the fictional telecommunication company *TELCO*, generated by *IBM* for training purposes:

- The outcome variable *Churn* indicates, if a customer departed within the last month (*Churn = Yes*) or not (*Churn = No*).
- Predictor variables contain:
 - Customers' *Gender* (*Female* or *Male*),
 - Customers' *SeniorCitizen* status (0 for no or 1 for yes),
 - Customers' *Tenure* with *TELCO* (month of membership), as well as
 - Customers' *MonthlyCharges* (in US-\$).

Real World Churn Analysis with Logistic Regression — the Data

```
DataChurn=import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-Presentations/d02e876360b32dd91d952f78791323d90f0a0395/TelcoChurnData.csv") %>%
  clean_names("upper_camel") %>%

select(Churn,Gender,SeniorCitizen,Tenure,MonthlyCharges) %>%
  mutate(Churn=factor(Churn,
levels=c("Yes","No")))
head(DataChurn)
```

Churn	Gender	SeniorCitizen	Tenure	MonthlyCharges
No	Female	0	1	29.85
No	Male	0	34	56.95
Yes	Male	0	2	53.85
No	Male	0	45	42.30
Yes	Female	0	2	70.70
Yes	Female	0	8	99.65

Real World Churn Analysis with Logistic Regression

— Do it yourself —

Create the Churn analysis with logistic regression. Click on the link in the footer to get an R-script with a skeleton for the analysis.

Download: `09-LogRegrExerc100.Rmd`

Results from Churn Analysis with
Logistic Regression

Confusion Matrix:

Prediction	Truth	
	Yes	No
Yes	239	150
No	322	1403

Accuracy:

.metric	.estimator	.estimate
accuracy	binary	0.7767266

.metric	.estimator	.estimate
sensitivity	binary	0.426025

Sensitivity:

.metric	.estimator	.estimate
specificity	binary	0.9034127

Specificity:

Hint: What do the column sums of the confusion matrix tell you?

Problem: Unballanced Training Data

Churn	n
Yes	1308
No	3621

Majority Class: *Churn = No* has 3621 observations in the training dataset.

Minority class *Churn = Yes* has 1308 observations in the training dataset.

What can we do?

Churn	n
Yes	1308
No	3621

- **Downsampling:** Randomly **delete observations from majority class** until ratio of the observations from the majority and the minority class reaches the desired ratio (e.g., 1:1).
- **Upsampling:** In simplest version, **creates new observations for the minority class** by copying randomly chosen observations from the minority class until the ratio of the observations from the majority and the minority class reaches the desired ratio (e.g., 1:1).
- Often, a combination of *downsampling* and *upsampling* is performed.

Performing Down-Sampling with `step_downsample()`

You need to add the R package `themis`. Then in your script, you can add

`step_downsample(Churn)` to the recipe (don't forget to execute the following command lines again). As a reminder our original `DataTrain` had 4,929 observations, $Churn_{Yes} = 1308$, $Churn_{No} = 3621$:

```
library(themis)
RecipeChurn=recipe(Churn ~ .,
data=DataTrain) %>%
  step_naomit() %>%
  step_dummy(Gender) %>%
  step_downsample(Churn)

# you do not need to do the
following steps.
# They just allow to display the
count() for the training data
ExtractedDataTrain=juice(RecipeChurn) %>% prep()
count(ExtractedDataTrain, Churn)
```

Churn	n
Yes	1308
No	1308

Note, the number of observations has decreased by 2313. This is an information loss!

Performing Up-Sampling with `step_upsample()`

You need to add the R package `themis`. Then in your script, you can add `step_upsample(Churn)` to the recipe (don't forget to execute the following command lines again). As a reminder our original `DataTrain` had 4,929 observations, $Churn_{Yes} = 1308$, $Churn_{No} = 3621$:

```
library(themis)
RecipeChurn=recipe(Churn ~ .,
data=DataTrain) %>%
  step_naomit() %>%
  step_dummy(Gender) %>%
  step_upsample(Churn)

# you do not need to do the
following steps.
# They just allow to display the
count() for the training data
ExtractedDataTrain=juice(RecipeChurn) %>% prep()
count(ExtractedDataTrain, Churn)
```

Churn	n
Yes	3621
No	3621

Note, the number of observations has increased by 2313. The information in the dataset has not increased!

Performing Up-Sampling with `step_smote()`. What is the advantage

As a reminder our original DataTrain had 4,929 observations, $Churn_{Yes} = 1308$, $Churn_{No} = 3621$:

```
library(themis)
RecipeChurn=recipe(Churn ~ .,
data=DataTrain) %>%
  step_naomit() %>%
  step_dummy(Gender) %>%
  step_upsample(Churn)

# you do not need to do the
following steps.
# They just allow to display the
count() for the training data
ExtractedDataTrain=juice(RecipeChurn
%>% prep())
count(ExtractedDataTrain, Churn)
```

Churn	n
Yes	3621
No	3621

Instead of copying a record from the training dataset, `step_smote()` finds the Nearest Neighbor to that record and creates a new record that has features generated as a weighted average between the Nearest Neighbor and the original record.

Download: 09-LogRegrExerc200.Rmd